

1. Introduction

Responsive web design is the practice of building websites that adapt seamlessly to different screen sizes and devices. This task focuses on mastering two powerful CSS layout systems — **CSS Flexbox** and **CSS Grid** — along with **Media Queries** to achieve a fully responsive, mobile-first design.

Flexbox is ideal for one-dimensional layouts (rows or columns), while CSS Grid excels at two-dimensional layouts. Together they form the backbone of modern frontend development.

2. Objective

- To understand and apply CSS Flexbox for one-dimensional alignment
- To implement CSS Grid for multi-column, two-dimensional layouts
- To use Media Queries for responsive breakpoints
- To build a mobile-first design approach
- To practice and master modern responsive frontend techniques

3. Tools & Deliverables

Category	Details
Tools Used	VS Code, Chrome DevTools, Browser
Dataset	UI Mockups / Design References
Deliverables	Responsive HTML/CSS Files, Screenshots
Outcome	Mobile-first responsive layout mastery

4. Key Concepts Covered

CSS Flexbox

- display: flex — enables flex container
- flex-direction: row / column
- justify-content: center / space-between
- align-items: center / flex-start
- flex-wrap: wrap for responsive rows
- gap property for spacing items

CSS Grid

- display: grid — enables grid container
- grid-template-columns / rows
- repeat() and fr unit for flexible sizing
- grid-gap / gap for gutters
- grid-column / grid-row span
- auto-fill and auto-fit for responsiveness

Media Queries

- @media (max-width: 768px) — tablet / large mobile breakpoint
- @media (max-width: 480px) — small mobile breakpoint
- Mobile-first approach: write base styles for mobile, scale up with min-width queries
- Collapse multi-column grids to single column on small screens

5. Implementation Steps

1 Step 1 – Project Setup

Created an HTML5 boilerplate with semantic tags: <header>, <nav>, <main>, <section>, <aside>, <footer>. Linked an external stylesheet (styles.css).

2 Step 2 – Navigation Bar with Flexbox

Built a responsive navbar using display:flex, justify-content:space-between, and align-items:center. Added flex-wrap:wrap so nav links stack on mobile screens.

3 Step 3 – Multi-Column Grid Layout

Implemented a 3-column card grid using grid-template-columns: repeat(3, 1fr). Used gap:20px for spacing. Cards stretch uniformly using align-items:stretch.

4 Step 4 – Sidebar + Content Layout

Used CSS Grid with grid-template-columns: 1fr 3fr to create a sidebar and main content area. Sidebar collapses to full width on mobile via media query.

5 Step 5 – Responsive Breakpoints

Added @media (max-width:768px) to switch 3-column grid to 1-column. Navigation stacks vertically. Font sizes and padding adjusted for readability.

6. Code Snippets

Flexbox Navbar

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  flex-wrap: wrap;  
  gap: 12px;  
}
```

CSS Grid – 3-Column Card Layout

```
.card-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}  
@media (max-width: 768px) {  
  .card-grid { grid-template-columns: 1fr; }  
}
```